LQCD Notebook Documentation

Ruiyu Ou / Univ. of NC - Chapel Hill

May 4, 2026

Contents

1 Introduction **3**
1.1 Purpose . . . 3
1.2 Scope . . . 3

2 System Requirements **3**
2.1 Software Dependencies . . . 3

3 Installation **3**
3.1 Clone Repository . . . 3
3.2 Install Dependencies . . . 3

4 Usage **4**
4.1 Initialization . . . 4
4.2 Sweep Fit . . . 4
4.2.1 Initialization . . . 4
4.2.2 Usage . . . 4
4.3 1 Fit . . . 4
4.3.1 Initialization . . . 4
4.3.2 Usage . . . 4
4.4 1 Plot . . . 4
4.4.1 Initialization . . . 4
4.4.2 Usage . . . 5
4.5 Summary Plot . . . 7
4.5.1 Initialization . . . 7
4.5.2 Usage . . . 8
4.6 Plain Text Fit Parameters and Quality Metrics . . . 8
4.6.1 Initialization . . . 8
4.6.2 Usage . . . 8

5 Example **8**

6 Code Differences Compared to Original Repository **8**
6.1 Code Behavior . . . 8
6.1.1 Directory & file structure . . . 8
6.2 New Code . . . 8

6.2.1 run_fit.py 8
6.2.2 json_priors_utils.py 8
6.3 Updated Code 8
6.3.1 nn_fit.py 8
6.3.2 summary_plot.py 15
6.3.3 plot_nn_stability_gevp.py 16

7 Troubleshooting **18**
7.1 Plotting Key Errors 18
7.2 Some Fit Errors 19

8 License **19**

9 Contact **20**

1 Introduction

This project is derived from the Cosmon Collaboration nn_c103_analysis repository, licensed under the BSD 3-Clause License. Additional modifications and development © 2026 Ruiyu Ou.

1.1 Purpose

This is a refactored version of nn_c103_analysis repository to be more user friendly for future use in hyperon or other LQCD analysis.

1.2 Scope

The current code encompasses all nn_bash script functionality into 1 code, but only allows fitting of deuterons, and plotting of GEVP comparison.

2 System Requirements

2.1 Software Dependencies

These dependencies are built on a conda environment.

- Python 3.12.13
- NumPy 1.26.4
- Scipy 1.17.1
- matplotlib 3.10.8
- Opt-Einsum 3.4.0
- Lsqfit 13.0.4
- gvar 13.1.7
- h5py 3.16.0

3 Installation

3.1 Clone Repository

```
git clone https://github.com/RuiyuOu/NN_Analysis_Luscher_GEVP.git
```

3.2 Install Dependencies

```
pip install \
    numpy==1.26.4 \
    scipy==1.17.1 \
    matplotlib==3.10.8 \
    opt-einsum==3.4.0 \
    lsqfit==13.0.4 \
```

```
    gvar==13.1.7 \
    h5py==3.16.0
```

4 Usage

The main notebook DeuteronGevp.ipynb contains the following functions.

4.1 Initialization

Initiate the notebook by choosing the appropriate python environment and proceed with *run all*, or running the *Initialization* block at the very top. From there you can proceed to any of the functions independently of one another.

4.2 Sweep Fit

4.2.1 Initialization

To initialize, run the cells in order:

- Interactive Functions
- Interactive Widgets
- RunSweepFitButton

4.2.2 Usage

4.3 1 Fit

4.3.1 Initialization

To initialize, run the cells in order:

- Interactive Functions
- Interactive Widgets
- RunFitButton

4.3.2 Usage

4.4 1 Plot

4.4.1 Initialization

To initialize, run the cells in order:

- Interactive Functions
- Interactive Widgets
- PlotButton

4.4.2 Usage

Plot Parameters Accordion Within the HTML display, the *Plot Parameters* accordion controls the plot in the background.

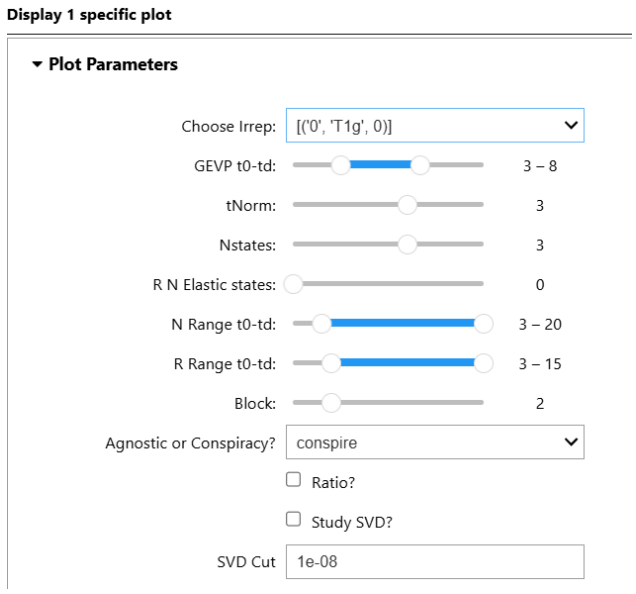
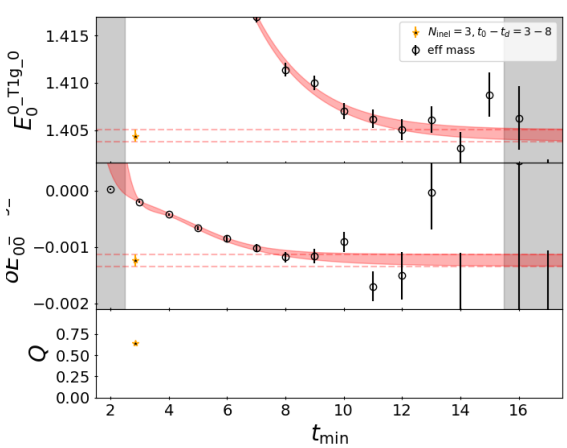


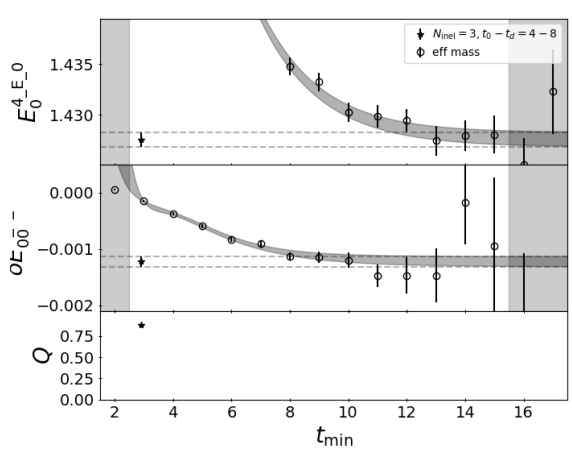
Figure 1: 1 Plot - Plot Parameters - Accordion display

One chooses the desired background fit by inputting the correct desired fit parameters. For all slider inputs, one can click the text next to it to directly edit in plain text. Make sure to hit enter to save the choice.

Example Observe: 2 different plots for the background with the parameters in the captions

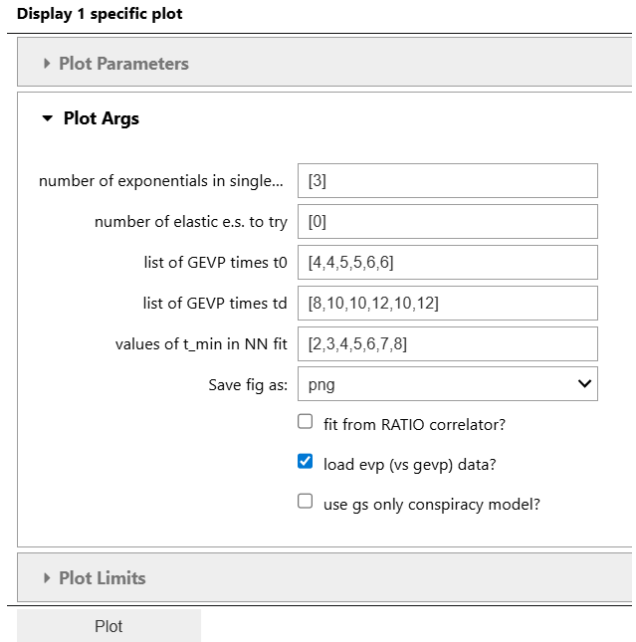


(a) 0T1g0 GEVP3-8 Block2 N3-20 R3-15 Nstates3



(b) 4E0 GEVP4-8 Block8 N3-20 R3-15 Nstates3

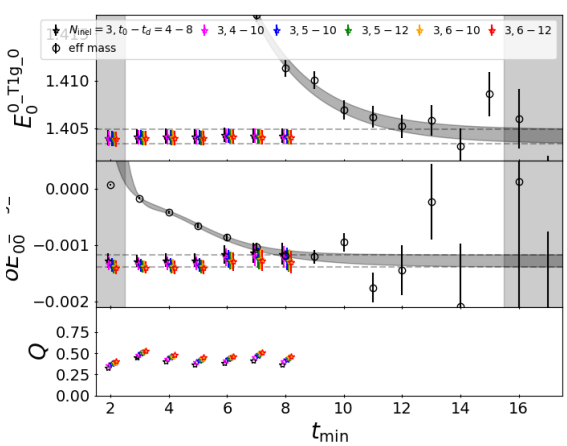
Plot Args Accordion Within the HTML display, the *Plot Args* accordion controls the scatter plot on the plot. Given the parameters shown in the figure, the scatter plot will contain fits with



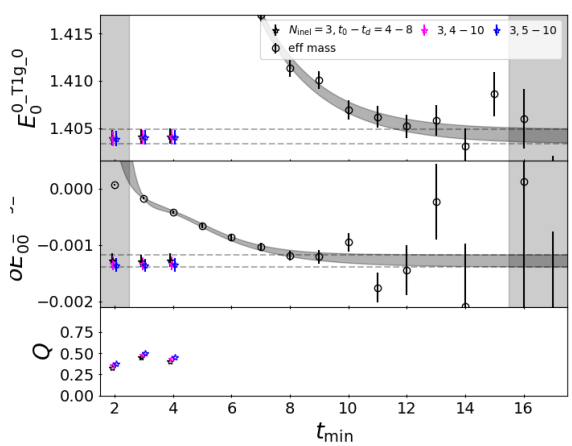
the following parameters.

- Any number of inelastic states given in the first box. example [3]
- Any number of elastic states given in the 2nd box. example [0]
- Any of the GEVP times given by t0-td in the same index of the array. example {4-8,4-10,5-10,5-12,6-10,6-12}
- Any of the min times for R fitting. example [2,3,4,5,6,7,8]

Example Observe: The same data plotted with different axis limits.



(a) GEVP times of {4-8,4-10,5-10,5-12,6-10,6-12}, min times of [2,3,4,5,6,7,8,9]



(b) GEVP times of {4-8,4-10,5-10}, min times of [2,3,4]

Plot Limits Accordion Within the HTML display, the *Plot Limits* accordion controls the plot axis limits. The default is all 0s, with the plot displaying the default axis limits if the upper and lower bounds are equivalent.

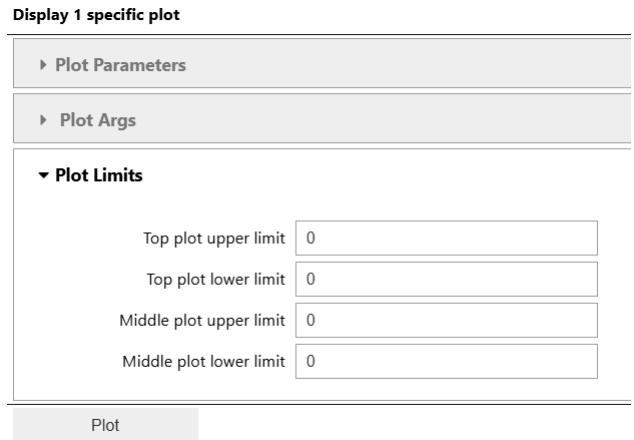
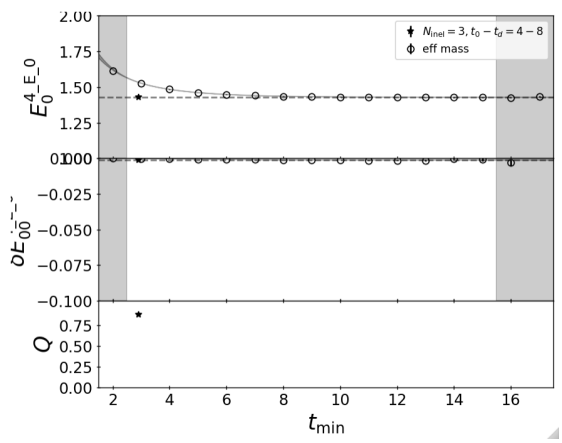
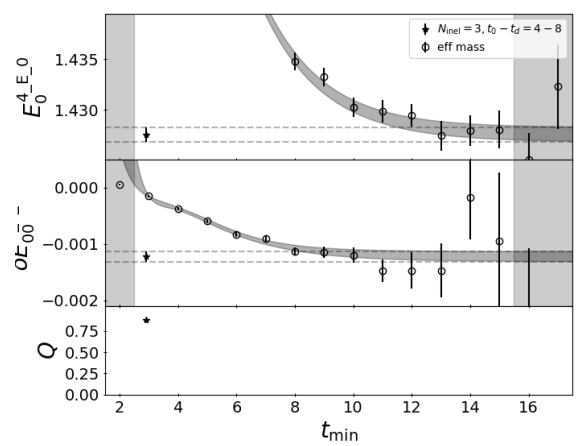


Figure 4: 1 Plot - Plot Limits - Accordion display

Example Observe: The same data plotted with different axis limits.



(a) Top plot axis: 2 - 1. Middle plot axis: 0 - -0.1.



(b) Default limits

4.5 Summary Plot

4.5.1 Initialization

To initialize, run the cells in order:

- Interactive Functions
- Interactive Widgets
- SummaryPlotButton

4.5.2 Usage

4.6 Plain Text Fit Parameters and Quality Metrics

4.6.1 Initialization

To initialize, run the cells in order:

- Interactive Functions
- Interactive Widgets
- PrintParamsButton

4.6.2 Usage

5 Example

6 Code Differences Compared to Original Repository

6.1 Code Behavior

6.1.1 Directory & file structure

Fit results now only contain 1 irrep. The fit .pickle file now lives in a subdirectory with the name of the irrep.

example filename

```
\result \0T1g0\NN _deuteron_tnorm2_t0-td_3-12_N_n4_t_3-20_NN_conspire_e0_t_3-13_ratio_False_block4.pickle
```

6.2 New Code

6.2.1 run_fit.py

This is a new code to fix the memory leak issue with sweep fitting.

6.2.2 json_priors_utils.py

This is a new code to save and read the new manual priors solution.

6.3 Updated Code

Every single line changed will be covered here.

How to read: Line number on the left. Line with a + infront is code added. Line with a - infront is code deleted.

6.3.1 nn_fit.py

1. New imports for functionality.

```
27    +import json_priors_utils as jsonpriors
28    +import pandas as pd
29    +
```

2. Parsing new parameters for functionality.

```
        if self.params['bootstrap']:
            self.boot0_file = self.filename.replace('_bsPrior-'+bs_prior,"")

        self.bspath = self.params["bspath"]

        self.data, self.irrep_dim = self.gevp_correlators()
        self.ratio_denom = self.get_ratio_combinations_Eff()
        self.ratio = self.params['ratio']
        #import IPython; IPython.embed()

        self.autopriors = self.params["autopriors"]


```

3. New code to accommodate the new directory structure of each irrep having their own directories in the results directory.

```
        self.ZjnSq = ZjnSq_irrep

        if self.params['Zjn_values']:
-           with h5.File(self.params['Zjn_values'],'w') as f5:

            IrrepPath = f"{self.params['masterkey'][0][0][0]}{self.params['masterkey'][0][0][1]}{self.params['masterkey'][0][0][2]}"

            outdir = f"result/{IrrepPath}"
            os.makedirs(outdir, exist_ok=True)

            with h5.File(f"result/{IrrepPath}/{self.params['Zjn_values']}",'w') as f5:
                for irrep in ZjnSq_irrep:
                    key = f"{irrep[0]}_{irrep[1]}"
                    for level in ZjnSq_irrep[irrep]:

```

4. Accommodating the new directory structure.

```
    def read_Zjn(self):
        ZjnSq_irrep = dict()
        E_sort      = dict()

        IrrepPath = f"{self.params['masterkey'][0][0][0]}{self.params['masterkey'][0][0][1]}{self.params['masterkey'][0][0][2]}"

        print('reading Zjn values')
-       print(f"    {self.params['Zjn_values']}")
-       with h5.File(self.params['Zjn_values'],'r') as f5:
        print(f"result/{IrrepPath}/{self.params['Zjn_values']}")
        with h5.File(f"result/{IrrepPath}/{self.params['Zjn_values']}",'r') as f5:
```

```
            #for irrep in f5.keys():
                #key = tuple(irrep.split('_'))
            for key in self.irreps:
```

5. Accommodating the new directory structure.

```
                    ZjnSq_irrep[key][int(level)] = f5[f"{irrep}/Zjn/{level}"][()]
                except:
                    print()
-                   print(f"your {self.params['Zjn_values']} is missing irreps")
-                   sys.exit(f"remake {self.params['Zjn_values']} with all irreps")
+                   print(f"your {f"result/{IrrepPath}/{self.params['Zjn_values']}"} is missing irreps")
+                   sys.exit(f"remake {f"result/{IrrepPath}/{self.params['Zjn_values']}"} with all irreps")
                E_sort[irrep] = f5[f"{irrep}/E_sort"][()]
        self.ZjnSq = ZjnSq_irrep
        self.E_sort = E_sort
```

6. Accommodating the new directory structure.

```
            E_tot = np.array(E_tot)
            i_sort = np.argsort(E_tot)
            print(i_sort)
-           with h5.File(self.params['Zjn_values'],'a') as f5:
+
+           IrrepPath = f"{self.params['masterkey'][0][0][0]}{self.params['masterkey'][0][0][1]}{self.params['masterkey'][0][0][2]}"
+
+           with h5.File(f"result/{IrrepPath}/{self.params['Zjn_values']}","a") as f5:
                key = f"{irrep[0]}_{irrep[1]}"
                f5.create_dataset(f"{key}/E_sort",data=i_sort)
```

7. Added for using the bootstrap data file functionality. It loads the .csv file from self.bspath which is declared by the input parameters.

```
                # if we have blocked - nucleon has the correct number of "configs"
                ncfg = nucleon[next(iter(nucleon))].shape[0]
                if self.old_bs:
-                   with h5.File('data/bslist_802.h5','r') as tmp5:
-                       self.draws = tmp5['bslist_802'][()]
+                   # with h5.File('data/bslist_802.h5','r') as tmp5:
+                   #     self.draws = tmp5['bslist_802'][()]
+                   print("Loading saved BS draws file")
+                   bspath = self.bspath
+                   self.draws = pd.read_csv(bspath).to_numpy(dtype=int)
```

```
665                 else:
666                     self.draws = bs_utils.make_bs_list(ncfg, self.
   params['Nbs_max'], seed=self.params['bs_seed'])
667                 self.h5_bs = True
```

8. Added for using the manual priors functionality. It loads the nn_priors.json file where the manual priors are stored. Line 879 is part of the auto priors function. Due to the priors being dependent on the number of states, it is currently designed to dump the priors when you first run a auto prior fit, then you may edit it as you would like and rerun the fit with manual priors.

```
876                                                 prior[(key, f"e_{n1}_{
   n2}")] = gv.gvar(en, sig_factor * e0)
877                                             prior[(key, f"z_{n1}_{n2
   }")] = gv.gvar(zn, 0.25)
878
879+                    jsonpriors.save_json_priors(prior,filename="
   nn_priors.json")
880+
881                     if self.params["debug"]:
882                         for k in prior:
883                             print(k, prior[k])
884+
885         elif type == "manual":
886-            pass
886+            print("Loading manual priors...")
887+            prior = jsonpriors.load_json_priors(filename="nn_priors.
   json")
888+            if self.params["debug"]:
889+                for k in prior:
890+                    print(k, prior[k])
891         return prior
892
893     def format_data(self, subset, nbs=0, ratio=True, svdcut=False):
```

9. self.autopriors takes on the value 'auto' or 'manual' which indicates the type of priors being fed.

```
1030                 if self.ratio:
1031                     x, y0, ybs = self.format_data(subset, nbs, svdcut
   =self.params['svd_study'])
1032                     prior = gv.BufferDict()
1033-                    prior = self.set_priors(prior, data=(x, y0), nbs=
   nbs, type="auto")
1033+                    prior = self.set_priors(prior, data=(x, y0), nbs=
   nbs, type=self.autopriors)
1034                 else:
1035                     # make Energy priors with ratio
1036                     x, y0, ybs = self.format_data(subset, nbs, ratio=
   True)
1037                     prior = gv.BufferDict()
1038-                    prior = self.set_priors(prior, data=(x, y0), nbs=
   nbs, type="auto")
```

```
1038+                    prior = self.set_priors(prior, data=(x, y0), nbs=
   nbs, type=self.autopriors)
1039
1040                     # remake data without ratio
1041                     x, y0, ybs = self.format_data(subset, nbs, ratio=
   False, svdcut=self.params['svd_study'])
1042                     # we need to make ground state overlap factor
   priors with non-ratio data
1043                     prior_z = gv.BufferDict()
1044-                    prior_z = self.set_priors(prior_z, data=(x, y0),
   nbs=nbs, type="auto")
1044+                    prior_z = self.set_priors(prior_z, data=(x, y0),
   nbs=nbs, type=self.autopriors)
1045                     for k in prior_z:
1046                         if k[-1] == 'z0':
1047                             # the single nucleon overlaps will be
   determined the same in both cases
```

10. Accommodating the fixed bootstrap file functionality.

```
1077                         print(e)
1078                         print('trying new BS prior_mean')
1079                         # I need to fix priors based upon ratio or
   not
1080-                        prior = self.set_priors(prior, data=(x, y0),
   nbs=nbs, type="auto", seed='2')
1080+                        prior = self.set_priors(prior, data=(x, y0),
   nbs=nbs, type=self.autopriors, seed='2')
1081                         result = lsqfit.nonlinear_fit(
1082                             data=(x, ybs), prior=prior, p0=p0_bs, fcn
   =self.func,
1083                             maxit=100000, fitter=self.params['fitter
   '], svdcut=svdcut
```

11. Accommodating the new directory structure.

```
1145             return
1146
1147         if self.params['bootstrap']:
1148-            if not os.path.exists(f"./result/{self.filename}_bs"):
1149-                gv.dump(self.bsresult, f"./result/{self.filename}_bs
   ")
1148+
1149+            IrrepPath = f"{self.params['masterkey'][0][0][0]}{self.
   params['masterkey'][0][0][1]}{self.params['masterkey'][0][0][2]}"
1150+
1151+            if not os.path.exists(f"./result/{IrrepPath}/{self.
   filename}_bs"):
1152+                gv.dump(self.bsresult, f"./result/{IrrepPath}/{self.
   filename}_bs")
1153             else:
1154-                bsresult = gv.load(f"./result/{self.filename}_bs")
```

```
1154+                 bsresult = gv.load(f"./result/{IrrepPath}/{self.
   filename}_bs")
1155                  for k in bsresult:
1156                      self.bsresult[k] = bsresult[k] + self.bsresult[k]
1157-                 gv.dump(self.bsresult, f"./result/{self.filename}_bs
   ")
1157+                 gv.dump(self.bsresult, f"./result/{IrrepPath}/{self.
   filename}_bs")
1158
1159             del self.bsresult
1160         else:
1161-            if os.path.exists(f"./result/{self.filename}"):
1162-                os.remove(f"./result/{self.filename}")
1161+
1162+            IrrepPath = f"{self.params['masterkey'][0][0][0]}{self.
   params['masterkey'][0][0][1]}{self.params['masterkey'][0][0][2]}"
1163+
1164+            if os.path.exists(f"./result/{IrrepPath}/{self.filename
   }"):
1165+                os.remove(f"./result/{IrrepPath}/{self.filename}")
1166             print('saving result')
1167-            gv.dump(self.posterior, f"./result/{self.filename}")
1167+            gv.dump(self.posterior, f"./result/{IrrepPath}/{self.
   filename}")
1168
1169
1170     def get_bs_pickle_Nbs(self):
1171-        if os.path.exists(f"./result/{self.filename}_bs"):
1172-            bsresult = gv.load(f"./result/{self.filename}_bs")
1171+        IrrepPath = f"{self.params['masterkey'][0][0][0]}{self.params
   ['masterkey'][0][0][1]}{self.params['masterkey'][0][0][2]}"
1172+
1173+        if os.path.exists(f"./result/{IrrepPath}/{self.filename}_bs")
   :
1174+            bsresult = gv.load(f"./result/{IrrepPath}/{self.filename}
   _bs")
1175             k = list(bsresult.keys())[0]
1176             Nbs=len(bsresult[k])-1 # first entry is boot0
1177         else:
```

12. Accommodating the new directory structure.

```
1179         return Nbs
1180
1181     def get_b0_posteriors(self):
1182-        if os.path.exists(f"./result/{self.boot0_file}"):
1183-            return gv.load(f"./result/{self.boot0_file}")
1182+        IrrepPath = f"{self.params['masterkey'][0][0][0]}{self.params
   ['masterkey'][0][0][1]}{self.params['masterkey'][0][0][2]}"
1183+        if os.path.exists(f"./result/{IrrepPath}/{self.boot0_file}"):
1184+            return gv.load(f"./result/{IrrepPath}/{self.boot0_file}")
1185         else:
1186-            print("can't get boot0, DOES NOT EXISTS: "+f"result/{self
```

```
   .boot0_file}")
1186+            print("can't get boot0, DOES NOT EXISTS: "+f"result/{
   IrrepPath}/{self.boot0_file}")
1187             sys.exit('run again with p["bootstrap"] = False')
```

13. Allow the nn_fit.py file to be able to run in the jupyter notebook with parameter input.

```
1425         return x, y
1426
1427
1428+def main(param):
1429+    #import IPython; IPython.embed() # insert this where you want to
   interact with fit in memory
1430+    for d in ['result', 'figures']:
1431+        if not os.path.exists(d):
1432+            os.makedirs(d)
1433+    fit = Fit(params=param)
1434+    bs_p = param
1435+    IrrepPath = f"{param['masterkey'][0][0][0]}{param['masterkey
   '][0][0][1]}{param['masterkey'][0][0][2]}"
1436+    if bs_p['get_Zj']:
1437+        if os.path.exists(f"result/{IrrepPath}/{bs_p['Zjn_values']}")
   :
1438+            # these two lines are just to generate self.irreps
1439+            fit.get_all_levels()
1440+            fit.restore_masterkey()
1441+            fit.read_Zjn()
1442+            ratio_denom = fit.get_ratio_combinations_Zjn()
1443+        else:
1444+            fit.get_all_levels()
1445+            fit.fit(n_start=0,ndraws=0)
1446+            fit.compute_Zjn()
1447+            fit.ratio_denom_Eff = dict(fit.ratio_denom)
1448+            ratio_denom = fit.get_ratio_combinations_Zjn()
1449+            # create Z_jn plots
1450+            if fit.params['show_Zjn']:
1451+                plt.ion()
1452+            fit.report_ZjnSq()
1453+            if fit.params['show_Zjn']:
1454+                plt.ioff()
1455+                plt.show()
1456+            # change back to requested fit info
1457+            fit.restore_masterkey()
1458+        # change NN ref states to match those from Zjn values
1459+        for k in ratio_denom:
1460+            if ratio_denom[k] != fit.ratio_denom[k]:
1461+                fit.ratio_denom[k] = ratio_denom[k]
1462+
1463+    if not bs_p['bootstrap']:
1464+        print('bs fits: boot0')
1465+        fit.fit(n_start=0,ndraws=0)
1466+        fit.save()
1467+
```

```
1468+    else:
1469+        bs_starts = bs_p['nbs_sub']* np.arange(bs_p['nbs']/bs_p['
   nbs_sub'],dtype=int)
1470+        bs_finished = fit.get_bs_pickle_Nbs()
1471+        for bs_start in bs_starts:
1472+            if bs_start < bs_finished:
1473+                print('bs fits:',bs_start+1,'->',bs_start+bs_p['
   nbs_sub'],'already done')
1474+            else:
1475+                print('bs fits:',bs_start+1,'->',bs_start+bs_p['
   nbs_sub'])
1476+                fit.fit(n_start=bs_start,ndraws=bs_p['nbs_sub'])
1477+                fit.save()
1478+
1479 if __name__ == "__main__":
1480     #import IPython; IPython.embed() # insert this where you want to
   interact with fit in memory
```

6.3.2 summary_plot.py

1. This is to accommodate the display of only 1 single irrep in the 1 plot procedure.

```
140         ('4', 'A1',  0):(1.4251,1.4395), ('4', 'A1',  1)
   :(1.4461,1.4605),
 }
141
142-def get_states(spin='deuteron', test=False):
143+def get_states(spin='deuteron', test=False, params=[]):
144     if spin == 'deuteron':
145-        states = [
146-                ('0', 'T1g', 0), ('0', 'T1g', 1),
147-                ('1', 'A2', 0),  ('1', 'E', 0),
148-                ('1', 'A2', 1),  ('1', 'E', 1),
149-                ('2', 'A2', 0),  ('2', 'B1', 0),  ('2', 'B2', 0),
150-                ('3', 'A2', 0),
151-                ('3', 'E', 0),
152-                ('4', 'A2', 0),  ('4', 'E', 0),
153-                ('4', 'A2', 1),  ('4', 'E', 1)
154-            ]
145+        if len(params) != 1:
146+            states = [
147+                    ('0', 'T1g', 0), ('0', 'T1g', 1),
148+                    ('1', 'A2', 0),  ('1', 'E', 0),
149+                    ('1', 'A2', 1),  ('1', 'E', 1),
150+                    ('2', 'A2', 0),  ('2', 'B1', 0),  ('2', 'B2', 0),
151+                    ('3', 'A2', 0),
152+                    ('3', 'E', 0),
153+                    ('4', 'A2', 0),  ('4', 'E', 0),
154+                    ('4', 'A2', 1),  ('4', 'E', 1)
155+                ]
156+        else:
157+            states = params
158+
159     elif spin == 'dineutron':
```

```
160         # (1,A1,2), (2,A1,1), (2, A1, 2), (3,A1,1), (3, A1, 2),
161         states = [
```

6.3.3 plot_nn_stability_gevp.py

1. New imports to accommodate the new directory structure and allowing input parameters into the function.

```
10 import nn_fit as fitter
11 import summary_plot
12
13+from pathlib import Path
14+
15 def flip(items, ncol):
16     return itertools.chain(*[items[i::ncol] for i in range(ncol)])
17
18-def main():
18+def main(params):
19     parser = argparse.ArgumentParser(
20         description='Plot results from NN fits with various correlator
   models')
21     # optimal fit
```

2. The addition of the summary plot argument to toggle the 4 summary plots

```
45         parser.add_argument('--debug',  default=False, action='
   store_true',
46                         help=       'add extra debug print statements?
   [%(default)s]')
47
48+    parser.add_argument('--summary',  default=False, action='store_true
   ',
49+                        help=       'display summary plots? [%(default)
   s]')
50+
51     args = parser.parse_args()
52     print(args)
```

3. Accommodating the new directory structure.

```
64
65     N_t = args.optimal.split('_NN')[0].split('_')[-1]
66
67-    nn_file  = 'NN_{nn_iso}_{t_norm}_t0-td_{gevp}_N_n{N_inel}_t_{N_t}'
67+    nn_file  = '{IrrepPath}/NN_{nn_iso}_{t_norm}_t0-td_{gevp}_N_n{
   N_inel}_t_{N_t}'
68     nn_file += '_NN_{nn_model}_e{nn_el}_t_{t0}-15_ratio_'+str(args.
   ratio)+block
69     if 'bsPrior' in args.optimal:
70         bsPrior = args.optimal.split('bsPrior-')[1].split('.')[0]
```

4. Allowing parameter inputs to plot 1 specific plot.

```
    if not os.path.exists("figures"):
        os.mkdir("figures")

-   states = summary_plot.get_states(nn_iso, args.test)
+   states = summary_plot.get_states(nn_iso, args.test, params)

    print('\nloading optimal fit:',args.optimal)
    post_optimal  = gv.load(args.optimal)
```

5. Accommodating the new directory structure for summary plotting.

```

    # get data keys
    fit_keys = {}
+
+   #Added to make the summary plotting work with the new filename system.
+   if args.summary:
+       OldToken    = Path(args.optimal).parts[1]
+       for q in states:
+           NewToken    = f'{q[0]}{q[1]}{q[2]}'
+           p           = Path(args.optimal)
+           parts       = list(p.parts)
+           parts[1]    = NewToken
+           NewPath     = str(Path(*parts))
+           if NewToken != OldToken:
+               post_optimal.update(gv.load(NewPath))
+
    for q in states:
        for k in post_optimal:
            if k[1] == 'e0' and k[0][1] == 'R' and k[0][0] == q:
```

6. Running the summary plot only whilst on a toggle.

```

    #import IPython; IPython.embed()
    # make summary plot
-   if nn_iso == 'deuteron':
-       mN = gevp_results['0_T1g_0']['E1'][0]
-   elif nn_iso == 'dineutron':
-       mN = gevp_results['0_A1g_0']['E1'][0]
-   # plot GEVP
-   summary_plot.summary_ENN(gevp_results, mN, gevp_lbls, color, spin=nn_iso,
-                            lbl0=r'GEVP: $t_0-t_d$=', fig=f"{nn_iso}_gevp_summary")
-   # plot tmin
-   summary_plot.summary_ENN(tmin_results, mN, tmin_lbls, t_color, spin=nn_iso,
-                            lbl0=r'$t_{\rm min}^{NN}=$', fig=f"{nn_iso}_tmin_summary")
```

```
227+    if args.summary:
228+        plt.show()
229+        plt.close('all')
230+        plt.ioff()
231+        if nn_iso == 'deuteron':
232+            mN = gevp_results['0_T1g_0']['E1'][0]
233+        elif nn_iso == 'dineutron':
234+            mN = gevp_results['0_A1g_0']['E1'][0]
235+        plt.close('all')
236+        # plot GEVP
237+        summary_plot.summary_ENN(gevp_results, mN, gevp_lbls, color,
   spin=nn_iso,
238+                                 lbl0=r'GEVP: $t_0-t_d$=', fig=f"{
   nn_iso}_gevp_summary")
239+        # plot tmin
240+        summary_plot.summary_ENN(tmin_results, mN, tmin_lbls, t_color,
    spin=nn_iso,
241+                                 lbl0=r'$t_{\rm min}^{NN}=$', fig=f"{
   nn_iso}_tmin_summary")
242
243-    plt.ioff()
244-    plt.show()
243
244 def plot_tmin(axnn, axnnR, axQ, state, models, arg, nnFile, nnDict,
   nnModel, optModel, fitKeys, nnData, r_gevp, l_gevp, r_tmin, l_tmin,
   ratio=True):
```

7. Accommodating the new directory structure.

```
333             nn_el     = models[model]['nn_el']
334             if 'agnostic' in fit_model:
335                 nn    = int(fit_model.split('agnostic_n')[1].split('_
   ')[0])
336+            IrrepPath = f"{state[0]}{state[1]}{state[2]}"
337             nnDict.update({'gevp':gevp, 'N_inel':n_inel, 'nn_el':nn_el
   , 't0':t,
338-                           'nn_model':models[model]['nn_model']})
338+                           'nn_model':models[model]['nn_model'],
339+                           'IrrepPath':IrrepPath})
340
341             if t == arg.tmin[0]:
342                 if gevp == arg.gevp[0]:
```

7 Troubleshooting

7.1 Plotting Key Errors

To generate the elements of the scatter plots, the display elements are hard coded into some plot file. Navigate to the corresponding erroneous dict and add the necessary key and value.

example One would like to run the summary plot function that will include the data with the fit start time of 10. However, a key error is raised in the summary_plot.py file in line 63.

```
    \summary_plot.py:63, in summary_ENN(all_results, mN, all_lbls, colors,
        lbl0, spin, fig, format)
     61 else:
     62     lbl = None
---> 63 color = colors[all_lbls[i]]
     64 ax.errorbar(irreps[irrep]+i-len(Ecm_mN)/2, e.mean, yerr=e.sdev,
     65             marker='s', linestyle='None', mfc='None',
     66             color=color, label=lbl)
     68 ax_dE.errorbar(irreps[irrep]+i-len(Ecm_mN)/2, dE_lab[i].mean, yerr
        =dE_lab[i].sdev,
     69             marker='s', linestyle='None', mfc='None',
     70             color=color, label=lbl)

KeyError: '10'
```

Backtracking the error brings one to plot_nn_stability_gevp.py file line 57 where the t_color dict defines the color of the scatter plot for the start times.

```
54        color = { '3-6' :'yellow', '3-8' :'r', '4-8' :'k', '4-10':'
   magenta',
55             '5-10':'b', '5-11': 'magenta','5-12':'g', '5-14':'yellow',
56             '6-10':'orange', '6-11':'firebrick', '6-12':'r','6-14':'
   firebrick'}
57    t_color = {'2':'gray', '3':'r', '4':'b', '5':'yellow', '6':'g',
   '7':'orange', '8':'magenta', '9':'k'}
```

One may then append the necessary '10':'color' to the dict to remove the key error and run the summary plot with all desired data.

7.2 Some Fit Errors

As additional parameters are created for the sake of bootstrap file saving, or manual priors. These parameters do not necessarily run without errors with the sweep fit functionality. Restarting the notebook should fix it.

example

```
    \nn_fit.py", line 1367, in twopoint_excited_states
    En2  = sum([np.exp(p[(key2, f"e{ni}")]) for ni in range(1, n2 + 1)])
                       ~^^^^^^^^^^^^^^^^^^^^
  File "src/gvar/_bufferdict.pyx", line 525, in gvar._bufferdict.
     BufferDict.__getitem__
KeyError: "undefined key: ((('0', 'T1g', 0), 'N', '0'), 'e3')"
```

This error is due to manual priors remaining turned from the 1 fit procedure.

8 License

BSD 3-Clause License

Copyright (c) 2023, Cosmon Collaboration Copyright (c) 2026, Ruiyu Ou All rights reserved.

Redistribution and use in source and binary forms, with or without modification, are permitted provided that the following conditions are met:

1. Redistributions of source code must retain the above copyright notice, this list of conditions and the following disclaimer.

2. Redistributions in binary form must reproduce the above copyright notice, this list of conditions and the following disclaimer in the documentation and/or other materials provided with the distribution.

3. Neither the name of the copyright holder nor the names of its contributors may be used to endorse or promote products derived from this software without specific prior written permission.

THIS SOFTWARE IS PROVIDED BY THE COPYRIGHT HOLDERS AND CONTRIBUTORS "AS IS" AND ANY EXPRESS OR IMPLIED WARRANTIES, INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR PURPOSE ARE DISCLAIMED. IN NO EVENT SHALL THE COPYRIGHT HOLDER OR CONTRIBUTORS BE LIABLE FOR ANY DIRECT, INDIRECT, INCIDENTAL, SPECIAL, EXEMPLARY, OR CONSEQUENTIAL DAMAGES.

9 Contact

Github: `https://github.com/RuiyuOu/NN_Analysis_Luscher_GEVP`